1. Mapa del sistema

Manual de organización y funcionamiento
Proyecto: sistema de facturación. Versión del código revisada: 11 de octubre de 2026. Esta guía explica dónde está cada responsabilidad y cómo se relacionan los archivos.

Qué hace el programa

Gestiona empleados, clientes, productos y facturas. Presenta la edad y el cumpleaños del empleado, controla días y horarios autorizados, genera facturas en PDF y prepara correos para clientes y alertas de acceso. El administrador supremo tiene funciones de gestión, pero también debe cumplir sus restricciones.

Cuatro capas que debes reconocer

Capa	Archivos	Responsabilidad
Pantalla	index.html y assets/css/estilos.css	Formularios, botones, paneles y presentación.
Navegador	assets/js/	Eventos, cálculos visibles y peticiones a PHP.
Servidor	api/ y config/	Validación, permisos, conexión y operaciones.
Datos	MySQL; scripts en database/	Información persistente y relaciones entre tablas.

El recorrido general

Persona → **pantalla** → **JavaScript** → **API PHP** → **MySQL** → **PHP** → **JavaScript** → **pantalla.** El navegador no se conecta directamente a MySQL. PHP actúa como intermediario.

Carpetas del proyecto

assets/: estilos, JavaScript y utilidades.
api/: lógica que responde a las peticiones.
config/: conexión y configuración del correo.
database/: creación y actualización de tablas.
vendor/PHPMailer/: biblioteca externa para enviar correos.
output/pdf/: material de estudio generado; no es la base de datos.

Cómo interpretar las rutas: todas las rutas relativas de este manual parten de C:/xampp/htdocs/facturacion. Por ejemplo, config/database.php significa C:/xampp/htdocs/facturacion/config/database.php.

Frase para tu evaluación: “Separé la presentación, la comunicación, los cálculos y las operaciones del servidor para ubicar con facilidad cada función”.

2. Conexión a la base de datos

El archivo exacto: config/database.php

La función **db()** crea una conexión con **PDO**, una herramienta de PHP para trabajar con bases de datos. api/index.php incluye este archivo con require_once y después ejecuta **$pdo = db();**

Nombre	Valor local predeterminado	Significado
DB_HOST	localhost	Servidor donde funciona MySQL.
DB_NAME	facturacion_db	Nombre de la base de datos.
DB_USER	root	Usuario de MySQL.
DB_PASS	Vacío	Contraseña local de MySQL.

Estos son valores predeterminados del código. Si el servidor tiene variables FACTURACION_DB_HOST, FACTURACION_DB_NAME, FACTURACION_DB_USER o FACTURACION_DB_PASS, sus valores tienen prioridad.

Qué significa la conexión PDO

mysql:host=...;dbname=...;charset=utf8mb4 indica el tipo de base de datos, el servidor, la base elegida y la codificación del texto. utf8mb4 permite manejar caracteres como tildes y ñ.

PDO::ERRMODE_EXCEPTION hace que un error de MySQL genere una excepción. El bloque catch registra el problema y responde con HTTP 503 y un mensaje de conexión para la pantalla.

Recorrido de una consulta

1. base-datos.js solicita una acción.
2. Apache ejecuta api/index.php.
3. require_once carga config/database.php.
4. db() devuelve la conexión en $pdo.
5. PHP utiliza $pdo para consultar o guardar.
6. La respuesta regresa como JSON.

Cómo se pasan los datos a SQL

prepare() prepara una consulta con marcadores como ?. **execute()** recibe los valores. **fetch()** obtiene una fila y **fetchAll()** obtiene varias. **lastInsertId()** devuelve el identificador del registro recién insertado.

Ejemplo del login: SELECT * FROM vendedores WHERE usuario = ? LIMIT 1. El nombre de usuario se entrega mediante execute([$user]), separado del texto SQL.

Configuración actual: el proyecto volvió al funcionamiento previo al cambio de .env. No hay un cargador de archivos .env activo. La conexión utiliza este archivo PHP y, opcionalmente, variables configuradas en el servidor.

3. Tablas y relaciones

Tabla	Qué conserva
vendedores	Nombre, usuario, clave protegida, nacimiento, rol, estado y restricciones.
clientes	Nombre, teléfono, correo, NIT y NRC.
productos	Código, descripción, precio y estado activo.
sucursales	Sucursales disponibles para las ventas.
facturas	Número, fecha, vendedor, sucursal, datos del cliente e importes.
factura_detalles	Cada artículo de una factura: cantidad, precio y total de línea.
accesos	Intentos rechazados, motivo, IP, navegador, fecha y estado del correo.
correos	Destinatario, asunto, cuerpo, adjunto, estado e intentos.
configuracion_correo	Destinatario de alertas, remitente y administrador que actualizó.

Cómo se relacionan

Un vendedor puede tener muchas facturas: facturas.vendedor_id identifica al empleado.
Una sucursal puede tener muchas facturas: facturas.sucursal_id identifica la sucursal.
Una factura tiene varios artículos: factura_detalles.factura_id enlaza cada línea con su factura.
Cada línea se refiere a un producto: factura_detalles.producto_id identifica el producto.

La factura guarda copias del nombre y los datos del cliente. El detalle conserva código, descripción y precio de la venta. Así se conserva la información usada al facturar aunque el catálogo cambie después.

Archivos SQL y herramienta de consulta

database/schema.sql: crea la estructura e incluye datos iniciales. Contiene instrucciones DROP TABLE, por lo que puede eliminar datos de una instalación existente.
database/actualizacion.sql: incorpora columnas y tablas para las funciones ampliadas.
database/correo-configuracion.sql: define la tabla de configuración del destinatario.
database/crear-admin.php: herramienta de consola para crear un administrador con contraseña protegida.

Para observar los datos locales, abre http://localhost/phpmyadmin/ y selecciona facturacion_db. Examinar una tabla te permite relacionar sus columnas con las consultas de PHP. No necesitas volver a importar los scripts para estudiar el sistema que ya funciona.

Respuesta breve: “facturas guarda el encabezado; factura_detalles guarda los artículos. Se relacionan por el identificador de la factura”.

4. Pantalla y comunicación

index.html y estilos.css

index.html contiene las vistas de login y panel, los formularios y las secciones de factura, historial, clientes, productos y administración. Cada elemento tiene un id que JavaScript utiliza para leer o mostrar datos.

assets/css/estilos.css define colores, tamaños, distribución, tablas, avisos y adaptación a distintas pantallas. El atributo hidden permite ocultar controles según la sección o el rol.

assets/js/base-datos.js

Define **API_URL = "api/index.php"** y el objeto **BaseDatos**. Su método solicitar() utiliza fetch para enviar una petición y leer el JSON de respuesta. Una URL como api/index.php?action=catalogs le dice a PHP qué operación ejecutar.

Método JavaScript	Acción PHP	Uso
login()	login	Enviar las credenciales.
getCatalogs()	catalogs	Obtener productos, sucursales y clientes.
getInvoices()	invoices	Consultar el historial permitido.
getInvoice()	invoice	Consultar una factura y sus detalles.
getProducts()	products	Obtener el catálogo de productos.
saveProduct()	save_product	Registrar un producto.
saveInvoice()	save_invoice	Guardar la factura.

Qué significan fetch, JSON y async

fetch: envía una solicitud HTTP.
JSON: formato con campos y valores para intercambiar datos.
async / await: permiten esperar la respuesta del servidor dentro de una función.
GET: consulta información. **POST:** envía datos para procesar una operación.

solicitar() conserva la cookie de sesión con credentials: same-origin y adjunta el token en X-CSRF-Token. Distingue un fallo de red, una respuesta que no es JSON y un error informado por PHP.

Para ejecutarlo: abre http://localhost/facturacion/ con Apache y MySQL activos. Abrir index.html como archivo o usar Live Server no ejecuta la API PHP de este sistema.

5. Inicio de sesión y seguridad

Quién participa en el login

principal.js → **iniciarSesion()** lee el formulario.
base-datos.js → **BaseDatos.login()** envía usuario y clave.
api/index.php → **case login** realiza las comprobaciones.
api/seguridad.php aporta las restricciones y alertas.

Pasos que realiza PHP

1. Exige una petición POST y datos JSON válidos.
2. Revisa si la IP acumula 10 accesos rechazados en los últimos 15 minutos.
3. Busca al usuario en vendedores.
4. Verifica la contraseña y las restricciones.
5. Si hay rechazo por credenciales o restricciones, registra una alerta.
6. Si se permite el acceso, renueva el identificador de sesión.
7. Guarda usuario_id y un token csrf en la sesión.
8. Devuelve los datos del empleado sin devolver su clave.

Contraseñas y sesión

password_hash() genera una representación protegida de una contraseña. **password_verify()** comprueba si una contraseña corresponde al hash. El login conserva compatibilidad con claves antiguas y las actualiza a un hash cuando corresponde, tras un acceso válido.

$_SESSION: información que PHP mantiene para reconocer al usuario. La cookie de sesión tiene HttpOnly y SameSite Strict; usa Secure cuando se ejecuta con HTTPS. logout destruye la sesión.

Comprobación en cada petición protegida

usuarioAutorizado($pdo) busca al empleado de la sesión y vuelve a comprobar sus restricciones. Para solicitudes POST, también verifica el token CSRF. Este token permite comprobar que el formulario corresponde a la sesión autenticada.

requerirSupremo() exige el rol supremo para operaciones administrativas. Un empleado no puede obtener esos permisos mostrando un botón oculto o cambiando valores del navegador.

Respuestas comunes: 401: falta iniciar sesión o las credenciales fallaron. 403: acción o acceso no permitido. 422: datos que no cumplen las validaciones. 429: demasiados intentos. 500: operación que no pudo completarse.

6. Paneles, horarios y cumpleaños

Panel personal y administrador

En **assets/js/paneles.js**, configurarPanel() ajusta el título y los controles según vendedorActual.rol. mostrarHorarioUsuario() muestra los días, las horas, las fechas y un aviso si el turno cruza la medianoche. El horario personal es de solo consulta.

mostrarAdministracion() solicita action=admin y presenta empleados, últimos accesos rechazados y últimos correos. editarEmpleado() carga un registro en el formulario. Los eventos submit envían los cambios a PHP.

Dónde se guardan empleados y restricciones

En **api/panel.php** → **save_user**, PHP comprueba nombre, usuario, nacimiento, días, horas, período, rol y contraseña. Permite crear o actualizar vendedores. No permite que el supremo se bloquee a sí mismo o quite su propio rol. Una nueva contraseña debe tener al menos 12 caracteres; un campo vacío durante la edición conserva la existente.

Reglas de acceso: motivoRestriccion()

Comprueba cuenta activa, fechas autorizadas, día y hora. Los días se guardan como números: 1=lunes, 2=martes, hasta 7=domingo. Las fechas límite son inclusivas. La hora inicial se permite y la final queda fuera del intervalo.

Ejemplo: para 08:00 a 18:00, entrar a las 08:00 está permitido y entrar a las 18:00 está fuera de horario. Para un turno nocturno, las horas posteriores a medianoche se vinculan con el día de inicio del turno. Estas reglas también se aplican al supremo.

Edad y mensaje de cumpleaños

assets/js/calculos/CalculoAntiguedad.js contiene la clase CalculoEdad. Valida la fecha y obtiene años, meses, semanas y días. El nombre del archivo es antiguo; la clase activa calcula la edad.

CalculoCumpleanos.js obtiene el aniversario y muestra si hoy cumple años, si todavía falta o si ya pasó este año, junto con los días hasta el próximo cumpleaños. Un nacimiento del 29 de febrero utiliza el 28 de febrero en años no bisiestos.

principal.js presenta esos resultados. **utilidades/fechas.js** obtiene la fecha de operación en America/El_Salvador y da formato a las fechas; **utilidades/formatos.js** formatea dinero para la pantalla.

7. Clientes, productos y factura

Clientes y productos

El formulario de clientes en paneles.js envía **save_client**. api/panel.php valida nombre y correo y guarda teléfono, NIT y NRC en clientes. cargarCatalogos() actualiza las opciones disponibles.

En principal.js, **guardarProducto()** utiliza BaseDatos.saveProduct(). api/index.php exige rol supremo, valida código, descripción y precio y evita códigos repetidos antes de insertar en productos.

Funciones que preparan la factura

Función de principal.js	Qué hace
mostrarSeccionNuevaFactura()	Prepara la sección y los datos de la nueva factura.
agregarItemFactura()	Valida cantidad y producto y añade una línea.
renderizarItemsFactura()	Presenta las líneas en la pantalla.
eliminarItem()	Quita una línea y actualiza los importes.
calcularTotalesFactura()	Usa CalculoFactura para mostrar base, IVA, total y comisión.
guardarFacturaBD()	Envía datos, informa el resultado y abre la descarga PDF.
mostrarHistorial()	Presenta las facturas permitidas y sus enlaces PDF.

Qué hace save_invoice en PHP

Obtiene el vendedor desde la sesión y la fecha del servidor. Comprueba que el cliente y la sucursal estén activos. Para tipo credito exige NIT y NRC. Valida que haya artículos y que las cantidades sean enteras entre 1 y 9999.

Consulta los precios reales de productos, recalcula cada línea y obtiene los importes oficiales. Así, un cambio de precio o vendedor enviado desde el navegador no determina lo que se guarda.

Transacción y correlativo definitivo

beginTransaction() inicia el grupo de operaciones. El código bloquea las filas de sucursales con FOR UPDATE y calcula el número FAC- seguido de ocho dígitos. Guarda facturas, factura_detalles, datos fiscales y el correo con PDF. commit() confirma; si ocurre un error dentro de la transacción, rollBack() revierte.

Después de confirmar la factura intenta enviar el correo. Un fallo de correo no deshace la venta. El correlativo que se muestra antes de guardar es orientativo; el servidor asigna el definitivo.

8. Cálculos y clases

IVA incluido: ejemplo de $79.00

Concepto	Cálculo	Resultado
Base sin IVA	79 ÷ 1.13	$69.91
IVA incluido	79 - 69.91	$9.09
Total	Base + IVA	$79.00
Comisión del vendedor	69.91 × 1%	$0.70

El precio del catálogo ya es el final. El IVA se desglosa y la comisión se calcula sobre la base sin IVA. Los resultados monetarios se redondean a dos decimales; la comisión no aumenta el total que paga el cliente.

Archivos en assets/js/calculos/

Clase / archivo	Responsabilidad
Calculo	Clase base con el método calcular().
CalculoMonetario	Aporta redondeo a dos decimales.
CalculoTotalLinea	Cantidad multiplicada por precio.
CalculoTotalFactura	Suma los totales de las líneas.
CalculoIVA	Extrae el IVA incluido del total.
CalculoSubtotal	Obtiene la base sin IVA restando el IVA.
CalculoComision	Aplica el porcentaje según la base sin IVA.
CalculoFactura	Combina total, IVA, base y comisión.
CalculoCorrelativo	Forma números consecutivos con prefijo y ceros.

Comisión: base hasta $100: 1%; más de $100 hasta $1,000: 2%; más de $1,000 hasta $2,000: 3%; más de $2,000: 4%. **api/calculos-factura.php** → **calcularImportesFactura()** realiza el cálculo oficial en PHP.

Programación orientada a objetos

Clase: define un comportamiento. **Objeto:** instancia creada con new. **Herencia:** una clase extiende otra. **Polimorfismo:** distintas clases implementan calcular() para operaciones diferentes. **Composición:** CalculoFactura crea y utiliza objetos de otros cálculos.

Archivos antiguos no cargados por index.html: CalculoImpuesto.js, CalculoTotalVenta.js y CalculoCorrelatico.js. Para estudiar el funcionamiento actual, sigue los scripts que sí se incluyen al final de index.html.

9. PDF y correos

Cómo se crea una factura PDF

api/factura-pdf.php → **pdfFactura($pdo, $id)** consulta la factura, el vendedor, la sucursal y los detalles. Construye un PDF con texto, páginas y estructura interna del documento. Incluye tipo de factura, datos del cliente, artículos, base, IVA y total.

api/panel.php → **pdf** comprueba que la factura exista y que el usuario pueda acceder a ella. Devuelve application/pdf y un encabezado de descarga. El empleado accede a sus facturas; el supremo puede acceder a las de otros empleados.

Enviar no es lo mismo que crear el PDF

vendor/PHPMailer/src/ contiene la biblioteca de correo. seguridad.php configura smtp.gmail.com, puerto 587, autenticación y STARTTLS. PHPMailer transporta el mensaje y el adjunto; pdfFactura crea el documento.

Archivos que configuran el correo

config/correo.php: carga remitente y clave desde variables del servidor o un archivo privado. La ruta privada local predeterminada es **C:/xampp/facturacion-privado/correo.php**, fuera de htdocs.
api/correo-configuracion.php: obtiene el destinatario guardado, muestra el estado de configuración y guarda cambios sin exponer la clave SMTP.
api/seguridad.php: contiene encolarCorreo(), enviarCorreo() y registrarAlerta().

Remitente, destinatario y alertas

El remitente es la cuenta Gmail que envía. El destinatario de alertas es el único correo elegido en el panel supremo. Cuando el maestro cambia ese correo, las alertas nuevas y pendientes pasan a su dirección, sin copias al destinatario anterior. Los correos de facturas siguen dirigidos a sus clientes.

La alerta registra usuario, motivo, fecha y hora, IP, navegador, acción y restricciones del empleado. Las credenciales incorrectas y los accesos fuera de las restricciones pueden generar estos avisos.

Estados y prueba del envío

pendiente: queda guardado sin completar el envío. **fallido:** el intento no pudo enviarse. **enviado:** el servidor aceptó el mensaje; comprueba recepción y spam. El panel permite probar y reintentar. No hay reintento automático programado.

El diagnóstico de Gmail comprueba dirección y longitud de 16 caracteres de la contraseña de aplicación; la prueba de envío confirma si el servicio la acepta. No incluyas contraseñas en tu exposición ni en GitHub.

10. Cómo estudiar y responder

Preguntas que te pueden hacer

Pregunta	Respuesta y ubicación
¿Dónde conecta MySQL?	config/database.php, función db(), mediante PDO. api/index.php la llama.
¿JavaScript guarda directamente?	Envía una petición con fetch; PHP valida y utiliza MySQL.
¿Dónde se verifica el horario?	api/seguridad.php, motivoRestriccion() y usuarioAutorizado().
¿Dónde cambio una contraseña?	Panel supremo; api/panel.php, save_user, utiliza password_hash().
¿Dónde se obtiene la edad?	CalculoAntiguedad.js, clase CalculoEdad; principal.js la presenta.
¿Dónde está el IVA?	CalculoIVA.js para pantalla; calcularImportesFactura() en PHP para guardar.
¿Dónde se guardan los artículos?	En factura_detalles, relacionados mediante factura_id.
¿Dónde se genera el PDF?	api/factura-pdf.php, función pdfFactura().
¿Dónde cambia el correo del maestro?	Panel supremo → save_mail_config → configuracion_correo.
¿Qué pasa si falla el correo?	La factura o alerta sigue guardada; el mensaje puede reintentarse.

Orden para practicar tu demostración

1. Enciende Apache y MySQL y abre http://localhost/facturacion/.
2. Inicia sesión dentro de los días y horas permitidos.
3. Muestra edad, cumpleaños y horario personal.
4. Explica el catálogo y registra un cliente si hace falta.
5. Prepara una factura y explica el IVA incluido con el ejemplo de $79.
6. Guarda, consulta el historial y descarga el PDF.
7. Con el supremo, muestra empleados, restricciones y destinatario de alertas.
8. Si Gmail está configurado, prueba el envío desde el panel.

Cómo localizar algo durante la evaluación

En tu editor utiliza Ctrl + F para buscar una función dentro del archivo, o la búsqueda global para encontrarla en el proyecto. Sigue primero la acción de la pantalla, después la función JavaScript, luego la acción PHP y finalmente la tabla.

Modelo de respuesta: “Esto sirve para guardar una factura. La pantalla llama a guardarFacturaBD() en principal.js; BaseDatos envía save_invoice; api/index.php valida, calcula y guarda en facturas y factura_detalles”.

No hace falta memorizar cada línea. Comprende qué entra, qué se valida, dónde se guarda y qué devuelve cada parte. Los comentarios en bloques te ayudan a ubicar las secciones.